

User Manual

Reader: a daily user — the deepest document, where the *why* lives **Claims last checked against the code:** 2026-09-16 — see *The documents, and what belongs in each* in [CLAUDE.md](#) .

Complete reference for gt's sixteen skills and its six modules. Written against **gt v0.15.0** (gt-wiki 0.2.1; gt-demo, gt-watch, gt-report-card, gt-farm and gt-flow 0.15.0).

The model

Claude forgets everything between sessions. The vault doesn't. Golden Thread's job is to move what you learn *out* of a session into a file whose **location** tells you how widely it applies — then load only the slice the next session needs.

```
session conversation
  ↓ gt-work
Projects/<slug>/memory/*.md      this project, this detail
  ↓ gt-promote
research.md                      dated findings – `unverified` allowed, if labelled
  ↓ gt-promote + gt-validate
decisions.md · design.md         settled choices – independently verified
  ↓ gt-promote + gt-validate
Knowledge/                       true for several projects – independently verified
  ↓ gt-promote + gt-validate
global-memory/                  true everywhere – independently verified
```

Facts move **up** as they prove general. They never move back down, and nothing is silently deleted.

Verification gates the climb — it is not a step at the end. `research.md` holds dated findings and accepts an `unverified` figure so long as it is labelled. Every level above it *reads as settled*, so a figure entering `decisions.md` , `design.md` , `Knowledge/` or `global-memory/` — or driving a production change — must be **independently verified**: re-derived by a validator that never saw the reasoning. A second pass by the same session is only `self-verified` ; it shares the assumptions that produced the error, so it confirms rather than checks. Run `/gt:gt-validate` *before* the promotion.

This rung was added after an unverified analysis reached `decisions.md` as an ADR recommending a four-ticker production change. An independent validator refuted three of the four — two of which would have deepened portfolio drawdown while appearing to improve their own ticker's. The ladder carried the error upward because nothing on it asked how the number was checked.

The discipline: write a fact at the narrowest scope that is honest, and promote it only when a second project proves it general. A fact promoted too early becomes a rule you must remember to disbelieve.

Loading is lazy, by design

`gt-open` reads the project's core docs and then reads `memory/MEMORY.md` **only** — an index of one line per file. The memory files themselves are read on demand.

A project with 70 notes costs ~80 lines to open instead of ~2,000. This is why `MEMORY.md` descriptions matter: they are the entire basis for deciding whether a file is worth opening.

Packs and the registry

New in 0.16.1.

Golden Thread keeps pluggable definitions in **slots** — how a language names things, which paths are noise, what a credential looks like, what a term means. Each slot is filled by **packs**: one JSON file of data, never a program.

```
gt_registry.py show naming --lang python    # what is in effect, and which pack it came from
gt_registry.py sources                     # every pack found, in precedence order
gt_registry.py slots                       # the slot table and merge modes
```

Precedence is community < core < local. A merged contribution *extends* coverage — a language core does not define — but never redefines what core defines; the user's own packs, under `<vault>/Projects/golden-thread/packs/`, always win. When one entry beats another the loser is listed as **SHADOWED** with the winner named, so a definition never disappears without a word. A pack that cannot be read is an error, not a silent gap.

Slots merge in one of two modes: **union** (entries accumulate and only identical ones collapse — **ignore**, **secrets**) or **map** (one value per key — **naming**, **encoding**, **filetype**).

Switching a definition off: **retract**. A union slot is additive, so nothing in it can be replaced — that rule is what stops a contributed pack retiring a core credential pattern. It also meant a noisy core definition could not be silenced, which is how a check stops being run at all. A pack in **your own vault** may therefore carry a **retract** list:

```
{"slot": "naming", "...": "...", "retract": [{"lang": "go"}]}
```

It matches on any field, so `{"lang": "go"}` switches off every Go definition in that slot at once — which is also how you choose which language packs are live, with no separate install step. Only vault packs may retract: a contributed or core pack declaring one is refused and reported. Every retraction prints as **RETRACTED**, alongside the shadowed entries, because a definition you turned off should still be visible to you.

Some slots have no consumer yet. `gt_registry.py slots` marks them. A pack in one of those resolves correctly and is then read by no tool at all — stated there rather than discovered.

Contributing a pack. gt runs no third-party code; packs are submitted, reviewed and merged into gt. `dev/submissions.py validate <pack>` checks a pack before a human reads it, and the release gate re-validates every shipped pack so review stays true rather than historical. A pack's tier is derived from whether its slot can reach model context, never from what the pack declares. See `SUBMISSIONS.md`.

What `/gt:gt-scan` can check, and how to ask

Fourteen languages ship with definitions. **Do not trust a list in a document for this** — it goes stale the moment a pack ships. Ask the machine you are on:

```
gt_scan_language.py --languages           # what this install can actually check
```

LANGUAGE	EXTENSIONS	NAMING CHECKED
python	<code>.py .pyi</code>	class, constant, function
javascript	<code>.js .mjs .cjs .jsx</code>	class, function, method
typescript	<code>.ts .tsx</code>	class, function, method, type
ruby	<code>.rb .rake Rakefile Gemfile</code>	class, function
php	<code>.php</code>	class, function
java	<code>.java</code>	class, type
csharp	<code>.cs</code>	class, type
rust	<code>.rs</code>	function, type
go	<code>.go</code>	function
shell	<code>.sh .bash .zsh</code>	function
sql, markdown, json, yaml		encoding only

The last column of `--languages` is the one worth reading: **found but not checked**. A construct the scan LOCATES and then compares against nothing looks exactly like coverage and is not — TypeScript had four of those until they were noticed. Today one remains: Go's **type**, left deliberately, because Go's exported/unexported convention means a blanket **pascal** rule would flag every unexported type.

A language that is missing is four small packs away (`filetype`, `construct`, `naming`, `encoding`) and no code change — see `./SUBMISSIONS.md`.

Vault layout

```
<vault>/
  CLAUDE.md          vault-wide conventions and page schema
  index.md           navigational index of Knowledge pages
  log.md             audit trail
  review-queue.md    items flagged for owner review
  lint-declines.md   suppressions, each with its reason

  Sources/           IMMUTABLE raw originals
  Knowledge/         cross-project wiki pages
  global-memory/     loaded in every session
    MEMORY.md        the index – this is what actually gets read

  Projects/
    README.md        master list + Dataview views
    CONVENTIONS.md   stages, properties, domain taxonomy, file roles
    PROTOCOL.md      cross-project process rules
    INFRASTRUCTURE.md the server fleet, defined ONCE

<slug>/
  README.md          status board + property frontmatter
  source.md          where the code lives + deploy plan
  idea.md            original brain dump – IMMUTABLE
  research.md        append-only dated findings
  decisions.md       append-only numbered ADRs
  design.md          current architecture
  spec.md            handoff artifact (when design is complete)
  runbook.md         operational procedures (optional)
  memory/
    MEMORY.md        index – the only thing gt-open reads
  <topic>.md         loaded on demand
```

Project properties

Every project `README.md` carries frontmatter. These are real Obsidian properties — they drive the tag pane, search, and the Dataview views.

```
---
type: project
slug: my-project      # must match the folder name
domain: trading       # coarse grouping
stage: active         # idea|research|design|active|complete|archived
topology: bastion-direct # local|remote|bastion-jump|bastion-direct|n/a
tags: [trading, platform, live]
parent: other-slug    # sub-projects only
---
```

Categorise with properties, not folders. Do not add a category level under `Projects/`. `gt-lint` walks projects to find `memory/` directories; a category folder is fine for *that* check since v0.3.0, but folders also fragment `gt-open`'s project lookup and buy nothing Obsidian doesn't already give you through properties. The one legitimate second level is a real sub-project created with `--parent`.

Source topology

`source.md` records **where the code actually lives**. It is read first by `gt-open`, because acting without it is how you edit the wrong box.

TOPOLOGY	MEANING
<code>local</code>	A folder on this machine. No deploy step.
<code>remote</code>	One repo, one server. Repo mirrors the prod structure; the server pulls from GitHub.
<code>bastion-jump</code>	Several servers, all reached through one gateway host.
<code>bastion-direct</code>	Several servers, each reachable independently.

Addressing is **role × env → host + path**, because one machine can serve several vhosts and the same role exists in more than one environment.

Bastion topologies also carry a **file plan**, marking each deployed file:

- **static** — byte-identical on every target. Deploy from one canonical copy.
- **unique** — a per-server variant. Never copy one host's version over another's.

That distinction is the whole point. Two files with the same name on two hosts may be the same file that must not drift, or different files that must not be merged — and nothing but the file plan tells you which.

The fleet itself is defined **once** in `Projects/INFRASTRUCTURE.md` and linked from each `source.md`. Never copy the host table into a project.

Typical use cases

Recipes for the situations that actually recur. Each names the command; the per-command detail follows in the sections below.

Starting something new

```
/gt:gt-create → work → /gt:gt-work
```

`gt-create` gathers slug, domain, topology and tags, scaffolds the folder, and writes `idea.md` from what you actually said.

`idea.md` is **immutable after that** — it is the traceable "why", and the one file you never rewrite when the plan changes. Close the session with `gt-work` or the session's findings die with it.

Picking up a project after time away

```
/gt:gt-open <slug>
```

Reads the project docs in dependency order and stops at `memory/MEMORY.md`. Skim the returned summary for **stage**, **hosts** and **blockers** before touching anything — a stale host entry is cheapest to correct before work starts. Then pull individual memory notes by

name as the work needs them.

"What should I work on?"

```
python3 <vault>/Projects/golden-thread/tools/gt_tasks.py # then read TASKS.md
```

Regenerate first, always. Effective priority is computed against the clock — a `p:: 1` older than seven days escalates, a `due` inside three days escalates, an open `pp_escalate` window escalates. Reading a stale `TASKS.md` gives you last week's ranking with this week's confidence.

`waiting:: user` is your list; `waiting:: agent` is the session's.

Two sections sit above the ranked lists. **Inbox** shows every unchecked line in `INBOX.md` — thoughts captured from anywhere and not yet filed — so nothing is lost and nothing unfiled ranks. **Review** shows projects the close-out probe thinks may be finished, with the reasons, so closing a project is asked about rather than forgotten (see *Closing a project*).

Tasks at `p:: 7` or higher are **shelved**: kept in the README as a record, excluded from every section and every escalation rule, counted in *Project standing* only.

Bringing an existing project into the vault

```
/gt:gt-ingest
```

Copies — never moves or deletes. If the notes cross-reference each other by `[[wikilink]]`, **count the links before renaming anything**; Obsidian resolves links by filename, so a tidy-up destroys the graph. Prefer keeping filenames.

You just learned something — where does it go?

```
/gt:gt-work      (end of session)
/gt:gt-promote   (when it proves general)
```

File at the **narrowest scope that is honest**. A finding that stopped changing graduates from `memory/` to `research.md`; a truth a *second* project hit graduates to `Knowledge/`. See "Where does this fact go?" below. Promotion is cheap; demotion is not.

A number is about to be recorded as fact

```
/gt:gt-validate
```

Before it lands in `decisions.md`, `design.md`, `Knowledge/` or `global-memory/`, and before any production change. The validator gets the claim, the rules and the artifact — never your reasoning. Budget for the possibility that it comes back **refuted**; that is the skill working, not failing.

Answering "how does this work?"

```
/gt:gt-query <topic>
```

Reads `index.md`, follows wikilinks into `Knowledge/`, then falls back to `grep` and project memory. If a page comes back `status: stale`, verify before acting on it.

Keeping the vault honest

```
/gt:gt-upgrade      # finish what install.sh could not apply (review, conflicts)
/gt:gt-doctor       # the whole install in one report; --fix re-wires hooks only
/gt:gt-lint         # broken links, orphans, unlisted memory, scope leaks
/gt:gt-optimize     # content that costs context and earns nothing back
/gt:gt-scan         # code against the language definitions in effect
/gt:gt-allin       # every check in one run; reports how many actually ran
/gt:gt-allin-commit # commit once the checks pass and a receipt covers the files
/gt:gt-context      # what the definitions here say, for a session to read
/gt:gt-validation   # what a validation established, and when it went stale
/gt:gt-handoff      # write the next session a handoff it can trust
/gt:gt-runbook-lint # facts duplicated across runbooks
/gt:gt-refresh      # upstream changes to Sources/
```

What `install.sh` does on an upgrade (since 0.14.0). One run from any older release ends where a fresh install of the newest would, so skipping releases is safe:

1. Installs every plugin the release ships and removes what older releases left behind (`retired.json`), after a backup.
2. Applies one-time **machine migrations** (`gt_machine_migrate.py`) — changes under `~/claude/` a skipped release would have made. Each runs once, judged from the machine's actual state; the first failure stops the install with `INSTALL INCOMPLETE`.
3. Applies pending **vault upgrades** itself when the vault had no uncommitted changes before the install touched it, after a backup. Results are left uncommitted for you to review. A vault the same install created gets an initial commit first. A vault holding your own uncommitted work is never touched — the install prints the command to run instead.

Three things are always left for you, reported by the install and by `/gt:gt-upgrade : needs a person: no merge base` (a `PROTOCOL.md` or `CONVENTIONS.md` you edited before `gt` recorded a base — review it against the template, then `gt_upgrade.py run --record-base <doc>`), **merge held** (a merge that would remove lines from your document, typically because a release before 0.14.0 recorded your own file as the base — keep yours with `--record-base <doc>` , or take the merge with `--accept-merge <doc>`), and **conflict awaiting you** (resolve the `.merge-conflict` file, then `--record-base`). None is ever merged unattended, and none is re-run on every install. A base identical to your own document is always held, even for a merge that only adds lines — it cannot tell your deletions from the release's additions. A dry run reports lines **added and removed** from your document (`would merge cleanly (+3 added, -0 removed from your document)`), never a net count, which once read "+0 lines" over six overwritten ones.

```
python3 $SCRIPTS/gt_machine_migrate.py status      # machine migrations pending here
python3 $SCRIPTS/gt_upgrade.py --vault <vault> status # vault upgrades pending
```

Modules (since 0.14.0). Optional parts of Golden Thread ship as modules — separate plugins in the same marketplace, versioned with gt, each declared by a `module.json`. 0.15.0 ships six:

MODULE	PLUGIN	DEFAULT	WHAT IT ADDS
wiki	gt-wiki	on	<code>/gt-wiki:gt-wiki</code> and four more skills for a standalone LLM wiki
demo	gt-demo	on	<code>/gt-demo:gt-demo</code> , the guided tour
watch	gt-watch	on	<code>/gt-watch:gt-watch</code> , a session-start hook and the <code>watch</code> setting (was <code>/gt:gt-watch</code>)
report-card	gt-report-card	on	the session report card: three hooks (PreCompact, SessionEnd, SessionStart) and the <code>report_card</code> and <code>closeout_check</code> settings; no command
farm	gt-farm	off for a fresh install	<code>/gt-farm:gt-farm</code> , work packets for an external AI (was <code>/gt:gt-farm</code>)
flow	gt-flow	on	<code>/gt-flow:gt-flow</code> , the flow view of the event stream

```

bash install.sh --list-modules      # each module, its state, and why
bash install.sh --without demo     # remove it; the choice is remembered
bash install.sh --with farm       # bring one in

```

Upgrading from 0.14.0. `/gt:gt-watch`, the report card and `/gt:gt-farm` were part of gt until 0.14.0; in 0.15.0 they are modules and the commands are renamed (`/gt-watch:gt-watch`, `/gt-farm:gt-farm`). The installer tells you where each went, once per command you had: `Moved: /gt:gt-watch → /gt-watch:gt-watch`, with (module `watch` is off: `./install.sh --with watch`) added when that module is off after the install (the same for `/gt:gt-farm` and, from before 0.14.0, `/gt:gt-demo`). One `install.sh` run over 0.14.0 ends where a fresh 0.15.0 install with the same module choices would. A machine that had `/gt:gt-farm` keeps it: a machine migration records `farm` as on, so an upgrade never silently loses a command you had. Your `watch`, `report_card` and `closeout_check` values are kept. Rolling back (`bash install.sh 0.14.0`) removes the module plugins 0.14.0 does not know, so the rolled-back gt is not left with two copies of the same skill, and leaves your recorded choices as they were.

The choice lives in `~/.claude/golden-thread/install-choices.json`. A module that is off leaves nothing behind: no plugin cache, marketplace entry, enabled flag, hooks or hook scripts. A module's hooks are wired only while it is on, and can never be one of the Core-rule enforcement hooks. `/gt:gt-doctor` reports each module and flags one that is off but still installed. A module may ship a demo act (`module.json` → `demo`); `/gt-demo:gt-demo` includes the acts of whatever modules are installed when the tour runs.

Run `gt-lint` after any structural change. Triage into three piles: *you broke it* (fix now), *already broken* (record in `review-queue.md`), *false positive* (suppress **with the reason**). The check that matters most is `memory-unlisted` — a memory file missing from `MEMORY.md` is a file Claude will never load.

Capturing a thought without leaving what you are doing

Add one checkbox line to `<vault>/INBOX.md`, from any session, in any project:

```

- [ ] Backtesting: try the nq_pullback set against a cold cache

```

No project, no priority, no date. Then go back to work. Later:

```
/gt:gt-review
```

reads the inbox (and daily notes, if you keep them), asks where each line belongs, files it as a task or a project, and checks the inbox line off with a `→ [[slug]]` pointer. An idea filed as a task gets `p:: 3` and no due date — a due date on an idea makes the deadline rule rank it above real work.

Closing a project

Delivery is not closure. A shipped project whose tasks stay open keeps escalating through its past due dates and outranks live work; on 2026-09-05 a talk delivered two days earlier held 24 of the top rows in the rollup.

```
python3 <vault>/Projects/golden-thread/tools/gt_closeout.py candidates
```

names projects whose signals say they may be finished: most open tasks past due, most tasks checked with nothing urgent left, three quiet weeks, or nothing open. The same probe runs at `/compact` and session end (setting `closeout_check`, from the report-card module), and `gt-work` asks after the last urgent task is checked. The question is always put to you, never acted on.

Closing means: `stage: complete`, leftover tasks to `p:: 7` (shelved, not deleted), `pp: 3`, a last `research.md` entry, and a look at what to promote. Every ask and every answer is recorded in `Projects/golden-thread/closeout-signals.jsonl`; `gt_closeout.py history` shows what "ready" has actually looked like for you, which is what the thresholds get tuned against.

Setup

`/gt:gt-init`

First run, or wiring a new machine. Writes `~/.claude/vault-config.json` (the pointer every other skill reads), scaffolds the vault, and adds a Golden Thread section to `~/.claude/CLAUDE.md`. Also installs Core-rule enforcement hooks into `~/.claude/settings.json` (`UserPromptSubmit` + `Stop` hooks) so that Core rules are actively asserted, not merely stored. Idempotent.

For a vault that predates the Core-rule tier, run the install step separately:

```
python3 $SCRIPTS/vault_init.py install-core-rules --vault ~/my-vault
```

`CLAUDE.md` is read at **session start**. A rule added mid-session does not apply until you restart — that looks like the rule being ignored.

What an install changes inside your vault

`install.sh` refreshes a few files in the vault on every run. Since 0.15.0 each is decided by **content**, and every change is named in the install output:

WHAT	RULE
Vault tools (<code>Projects/golden-thread/tools/*.py</code>)	Replaced only when your copy is text gt shipped (listed in <code>templates/shipped-hashes.json</code>). Anything else is kept : <code>Vault tool MODIFIED LOCALLY → ... kept</code> , with the template to diff against.
Git hooks (<code>.githooks/*</code>)	Same rule as vault tools: a copy gt shipped is updated; a hook matching no shipped version (your edit, committed or not) is kept and reported as modified locally.
<code>core.hooksPath</code>	Set to <code>.githooks</code> only when unset. If you set your own, it is left alone and the install says how to call the attribution hooks from yours.
Core rules and CLAUDE.md's "First: is enforcement active?" section	A missing rule is added back and named (<code>Added Core rule → ...</code>); so is the section.
<code>log.md</code> , each <code>decisions.md</code>	Converted once to generated files (the 0.11.0 spool migration) when the vault was clean before the install; each converted file is named, with any note (such as trailing blank lines dropped).

Before the first of these writes, the files involved are backed up to `~/.claude/golden-thread/backups/install-vault-files-<stamp>.tar.gz` (with the previous `core.hooksPath`). The backup is deleted again when the install changed none of them.

Keeping something removed. A rule or section you delete on purpose comes back on the next install unless you list it in `.gt-removed` , beside `core-rules/` (by default `Projects/golden-thread/.gt-removed`). One entry per line, `#` for comments:

```
core_parallel_when_beneficial.md    # a Core rule file name
claude-md-enforcement-section      # CLAUDE.md's "is enforcement active?" section
```

Listed items are never re-created (`Left removed → ...`). gt never writes this file. Removing a rule file does not switch off a hook that enforces it (`enforcement: validated`); the install warns when that is the case — use `gt_settings.py` for that.

`/gt:gt-create`

Scaffolds a project. Gathers slug, title, tags, **domain**, sub-project parent, runbook, and **topology**; runs the script; then fills `idea.md` from what you actually said. `idea.md` is immutable afterwards — it is the traceable "why".

Writing to the shared files

`log.md` , `decisions.md` and `TASKS.md` are **generated**. They are the files every session writes and none owns, and in one working tree that means last-writer-wins with no conflict marker to warn you. So none of them is written directly.

```
T=<vault>/Projects/golden-thread/tools

# A log entry. Writes only this session's spool file, then re-renders log.md.
python3 $T/gt_log.py --vault <vault> add "2026-01-01 10:00 CST [work] my-project - what happened"

# An ADR. Reserve the number FIRST -- two sessions that both read "the highest is 5"
# will both write ADR-6. This takes the number in a single atomic step and prints it.
python3 $T/gt_adr.py --vault <vault> allocate my-project --title "The choice"
# -> 7 (and creates the file that holds ADR-7; write the body into it)
python3 $T/gt_adr.py --vault <vault> merge my-project

# Who has spooled what, and any allocation left unfinished
python3 $T/gt_log.py status
python3 $T/gt_adr.py status my-project
```

Adopting this in an existing vault is one command per file. It freezes what is there as a baseline that sorts first, so no history is rewritten and no ADR is renumbered, and it refuses unless the merge reproduces the original byte for byte:

```
python3 $T/gt_log.py --vault <vault> migrate
python3 $T/gt_adr.py --vault <vault> migrate my-project
```

`migrate` refuses on a project whose ADR numbers already collide, because renumbering would invalidate every existing reference to them. `gt-lint`'s `adr-collision` reports those so you can decide. A deliberate `## ADR-6 amendment:` is not a collision and is left alone.

Movement events. `gt_events.py` records *what moved where* as structured JSON lines — the data `/gt-flow:gt-flow` draws. Same spool pattern as the log: one file per session under `spool/events/`, merged into `events.jsonl`. The tool and schema (v1) shipped in 0.13.0; since 0.15.0 operations record events as they happen:

- `gt_log.py add "<line>" --event <kind> --item <path> [--from --to --level-from --level-to --project]` spools the log line and one event in one command. `/gt:gt-promote` and `/gt:gt-refresh` record their moves this way; `/gt:gt-review`, `/gt:gt-work` and `/gt:gt-ingest` emit theirs with `gt_events.py emit`.
- `gt_adr.py allocate` emits an `adr` event for each number it reserves.
- `vault_init.py` emits `create`, `rename`, `merge` and `archive` when `create-project`, `rename-project`, `merge-project` and `archive-project` actually do something (never under `--dry-run`). `archive-project` is the only source of `archive`: `gt_closeout.py answer <slug> yes` records your decision to close, not the move, and emits no event.
- `gt_tasks.py` emits `task.open` / `task.done` for checkbox lines that changed — once the stream has been seeded with task events by a backfill.

An event can never fail the operation it describes: a failure is one line on stderr.

A vault older than the events has none. `backfill` rebuilds the history from git and `log.md` — Knowledge and global-memory additions, renames between levels, project renames, README checkbox changes — as `actor: backfill` events. Preview it first; a second run adds nothing. (A dry run on the author's vault found about 1,430 events.)

```
python3 $T/gt_log.py --vault <vault> add "2026-01-01 [graduate] quote_api → Knowledge" --event promote \
  --item Knowledge/Quote-API.md --from Projects/ats/memory/quote_api.md \
  --to Knowledge/Quote-API.md --level-from 2 --level-to 4 --project ats
python3 $T/gt_events.py --vault <vault> backfill --dry-run # what history would be added
python3 $T/gt_events.py --vault <vault> validate # report bad lines, exit 1 if any
python3 $T/gt_events.py --vault <vault> list --kind promote --json
```

Every event is checked before it is written: vault-relative paths only, a closed list of kinds, levels 1–5, a note of at most 120 characters, no unknown keys.

Hand-editing a generated file is still the wrong place for it, but it is no longer lost (0.15.0): a line typed into `log.md` is captured into `spool/log/hand-edits-<time>.md` at the next merge and announced — a line starting with a date sorts by it, an undated one lands after the baseline — and `gt_adr.py merge` refuses to render over a hand-written line in `decisions.md`, naming it, until you move it into an ADR. `gt-lint` reports the file as `generated-hand-edited`.

Both commands sit on one shared primitive, `gt_spool.py`: per-session spool files plus a merge whose ordering uses nothing machine-specific, so two machines rendering the same spools produce byte-identical output. Bytes that are not valid UTF-8 (an old latin-1 `é` in `log.md` or `decisions.md`) pass through `migrate` and `merge` exactly as they were (0.15.0). You do not call it directly; it is documented here because a merge that ordered by mtime or directory order would make the *generated* file conflict in git, which is worse than the problem being solved.

Daily work

`/gt:gt-open <slug>`

Loads a project and stops. It is **read-only** — nothing is written during loading.

Reads, in order: `CONVENTIONS.md` and `PROTOCOL.md` (once per session), the project `README.md` — frontmatter first, since `domain / stage / topology / tags` is the fastest read of where the project stands — then `source.md` → `idea.md` → `research.md` → `decisions.md` → `design.md` → `spec.md` → `runbook.md`, the linked fleet page, and finally `memory/MEMORY.md` — **the index only**. Individual memory files load on demand.

`source.md` leads the project docs deliberately. It says which box serves which role in which environment. Acting before reading it is how you edit the wrong host, or overwrite a file that exists on three machines.

`research.md` over ~200 lines is read by its headings, plus the entries relevant to the work at hand and the most recent few. It is append-only and grows without bound; reading it whole crowds out the context the actual work needs.

Sub-projects load only when you name one, or the status board shows it as the active item.

Announces the `review-queue.md` count once, then summarises stage, **topology and hosts**, next action, blockers, and what memory exists but is unloaded — the host list so you can correct a stale entry *before* work starts rather than after.

`/gt:gt-route`

The middle of a session. `gt-open` runs before the session knows what it is; `gt-work` runs after it is over. Between them sits the part where the work actually changes shape, and nothing served it.

Run it whenever the ground has shifted, or whenever you cannot name what you are doing. It reads only the conversation already in context — never project files, because reading `research.md` to decide where a note goes costs more than the note — and answers

four things:

1. **What this has become.** Restated in a sentence, with the drift named explicitly if the session no longer resembles what it opened with. This is most of the value: the drift is usually felt before it is articulated.
2. **Where the output belongs** — the [Where does this fact go?](#) table, applied to the material actually in hand, naming more than one destination when more than one is true.
3. **Whether the work is in the right place** — project loaded, harness (a real PTY belongs in a terminal; vault context belongs in Claudian), and whether the model matches the difficulty. It reports mismatches; it changes nothing.
4. **One next action.** A skill to invoke or a file to append to — not a menu.

It writes nothing, with a single exception: one `INBOX.md` checkbox line, which is what `INBOX.md` is for. Everything else is handed to the skill that owns the destination, which does the writing under its own claim.

"Nothing, keep going" is a valid answer and the skill is built to give it. A check that always finds work is a check nobody runs twice.

Use it when a session opened as one thing and became another; when a finding might belong to a project other than the one loaded; when you are about to write something down and are not sure which file; or when you simply feel lost. Being wrong costs one re-run.

`/gt:gt-work`

End of session. The step people skip, and skipping it is what makes the vault decay.

GOES TO	WHAT BELONGS THERE
<code>research.md</code>	Dated findings and gotchas. New discoveries only.
<code>decisions.md</code>	Numbered ADR: decision, context, rejected alternatives. Stable choices only.
<code>design.md</code>	Rewritten in place if architecture changed. Describes <i>now</i> .
<code>memory/*.md</code>	Session state, updated in place.
<code>spec.md</code>	Created when design is complete enough to hand off.

Memory files carry `level` and `enforcement` in frontmatter to make a rule's durability explicit:

```
metadata:
  type: core | feedback | user | reference
  level: core | context | generic      # default: generic
  enforcement: validated | reminder    # required only for level: core
```

Default to `generic`. Do not set `level: core` here — Core is a promotion, and it requires wiring an enforcement hook, which is `gt-promote`'s job.

After writing, `gt-work` actively asks whether any finding applies beyond this project and should go to `Knowledge/`, `global-memory/`, or a new project.

Also updates the `stage:`, `topology:`, and `domain:` properties in `README.md` when the project's reality changes.

Not in `decisions.md` : anything you might change next session. An ADR you reverse next week teaches the vault to lie to you.

/gt:gt-query

Reads `index.md`, follows wikilinks 2–3 hops into `Knowledge/`, falls back to `grep`, then project memory. Be specific. If a result is `status: stale`, verify it.

/gt:gt-ingest

Bulk-imports an existing project. Copies — never moves or deletes.

The trap: if the notes cross-reference each other by `[[wikilink]]`, renaming or merging files destroys that graph, because Obsidian resolves links by filename. Measure first:

```
grep -roh '\[ \[ \[ ^ \] * \] \]' --include='*.md' . | wc -l
```

Prefer keeping filenames. If you must repoint, match the *whole* link — `[[name]]` and `[[name|alias]]` — never a substring, or `[[foo]]` will corrupt `[[foo_bar]]`. Never repoint links inside `Sources/`.

/gt:gt-review

Empties `INBOX.md`: reads every unchecked line (and daily notes, if configured), asks where each belongs, files it as a task or project, and checks the inbox line off with a pointer. Regenerates the rollup at the end.

Context management

/gt-farm:gt-farm

Module `farm` (plugin `gt-farm`). Off for a fresh install — `bash install.sh --with farm` adds it. A machine upgrading from a `gt` that shipped `/gt:gt-farm` keeps it on.

Routes work out of this context to an external AI service when work is bulk, mechanical, or genuinely benefits from a non-Claude second opinion.

The unit is a **work packet**: self-contained, with a strict return contract (`FINDINGS` + `GAPS` blocks). The transport is swappable — the user pastes it into a web UI today; a script sends it to an API later. The packet is identical either way.

All four gates must pass before a task leaves. Any failure means the work stays here.

GATE	QUESTION
Stateless	Answerable with no vault or repo state?
Self-contained	Fits in a paragraph plus a list of URLs?
Checkable	Verifiable without redoing the work?
Releasable	Every input safe to hand a third party?

Releasable is default-deny. Never include: account identifiers, balances, hostnames, internal IPs, credential locations, non-public source code, or personal identifiers.

Key rules: - One packet, one task. A packet asking three questions returns three half-answers. - Supply a vocabulary block with exact model IDs, product names, and version numbers read from the system — never from memory. - Results come back as `unverified` . Promote a finding only after opening its `source:` URL and confirming the claim. - `GAPS` is mandatory and `GAPS: None` must be justified, not asserted. - Cite the specific page carrying the claim, never a homepage.

Packets are saved to `<packet_dir>/<YYYY-MM-DD>-<slug>.md` . The packet dir is `farm_packet_dir` in `~/.claude/vault-config.json` if set, else `Projects/<current project>/packets/` . A relative value is taken from the vault root and a literal `<project>` in it becomes the current project's slug; an absolute value is used as is. The skill names no particular AI service: which services you have is your configuration (`farm_services` in `vault-config.json` names a table of them). Copying a packet uses whichever clipboard tool the machine has — `pbcopy` , `wl-copy` , `xclip` or `clip.exe` — and with none, you are given the packet file's path instead.

Knowledge management

/gt:gt-promote

The judgement call: **has a second project proved this general?**

FROM → TO	WHEN
memory/ → research / decisions	The finding stopped changing session to session
project → Knowledge/	A second project hit the same truth
Knowledge/ → global-memory/	Needed in every session, every project
anywhere → new project	The idea doesn't belong where it is

Nothing is deleted. Retiring sets `status: stale` and records what superseded it.

Promoting a rule to Core (`level: core`) also wires the enforcement hook — `gt-promote` handles this; `gt-work` does not.

/gt:gt-refresh

Checks `Sources/` for upstream changes. Supersedes with a **new** immutable file carrying `supersedes:` — never edits the old one.

Verification

/gt:gt-validate

Verify a claim by **re-deriving** it, not by reviewing it. Use before a finding is recorded as fact, before a production change, or whenever a number matters.

The one rule: the validator never receives the reasoning that produced the claim. Give it the reasoning and it grades the argument — inheriting the same blind spot. Give it only the claim, the rules and the artifact, and it has to go back to primary sources. That is the only thing that catches a wrong premise.

Reduce the work to a **single falsifiable assertion**. "The analysis is sound" is not validatable; "widening NQ's target to 1.25 improves risk-adjusted return" is. Validate several claims **separately** — a bundled claim returns a bundled verdict, which hides which part failed.

Loads the project's `validation-rules.md` , and the parent's too, since packs are inherited. Those are standing invariants the validator enforces whether or not the request mentions them — because *a requester who has already made a domain error will not think to ask the validator to check for it*. On conflict, the pack wins.

CLASS	USE WHEN THE RISK IS
empirical	A number, measurement or result could be wrong
vantage	The measuring position may not be able to observe the answer
rule-compliance	Work must satisfy <code>core-rules/</code> or <code>CONVENTIONS.md</code>
code	Code may not do what its name, comment or docs claim

Pick every class whose risk is present. **Do not substitute one generic reviewer** — that produces agreement, not verification.

The packet is exactly three fields — `claim` , `rules` , `artifact` — then re-read it and strip your conclusions, your numbers, your confidence, and any adjective implying the expected answer. "Confirm that X" becomes "determine whether X". Withholding *reasoning* is isolation; withholding *access* is just a broken validator, so keep the hostnames, API shapes and filters it needs to reach the artifact alone.

Run the validators **independently**, and read their derivation *before* re-reading your own — reading yours first turns the exercise back into review.

Three verdicts: **confirmed**, **refuted** (quantify the divergence), and **cannot-verify** (name what was missing).

Append the verdict to `research.md` . A refutation supersedes the original finding.

cannot-verify is never a pass. A validator that could not check something and stayed quiet manufactures false assurance — worse than no validator at all.

You are the worst possible author of this packet, because you already know the answer and your framing leaks. Treat building it as an adversarial exercise against yourself. A validator disagreeing is a *result*, not a failure.

Maintenance

/gt:gt-lint

CHECK	CATCHES
index-gap	Knowledge page not in <code>index.md</code> — <code>gt-query</code> won't find it
broken-link	<code>[[wikilink]]</code> resolving to nothing
orphan	Knowledge page linked from nowhere
memory-unlisted	<code>memory/*.md</code> not in <code>MEMORY.md</code> — Claude will never load it
global-gap	<code>global-memory/*.md</code> not in its index
memory-bloat	<code>global-memory</code> file over 30 lines
global-scope-leak	Project-specific content in <code>global-memory/</code>
superseded-cited	Knowledge page citing a superseded source
stale	Knowledge page marked <code>status: stale</code>
source-todo	<code>source.md</code> with no topology or deployment targets
frontmatter	Project README missing properties, or slug ≠ folder
core-misplaced	A rule declares <code>level: core</code> but lives outside <code>core-rules/</code>
core-no-enforcement	<code>level: core</code> with no <code>enforcement</code> field declared
core-unenforced	The declared enforcement hook is not actually wired

`core-unenforced` is the critical one — it is the machine-checkable form of "rule stored but never asserted". Treat it as a real defect. Do not suppress it — suppressing `core-unenforced` recreates the original bug with a paper trail saying it was fine. Fix: `vault_init.py install-core-rules --vault <vault>`.

Declines go in `lint-declines.md` as `suppress: with the reason`.

Triage findings into three piles: *you broke it* (fix now), *already broken* (record in `review-queue.md` — don't guess at a target), *false positive* (suppress with reason).

/gt:gt-settings

View and change what Golden Thread does automatically. Every automatic behaviour is registered here and can be switched off.

SETTING	VALUES	DEFAULT	WHAT IT DOES
<code>component_updates</code>	<code>off</code> · <code>report</code> · <code>confirm</code> · <code>auto</code>	<code>report</code>	At session start, compares installed hooks/scripts against plugin source and reports drift
<code>version_check</code>	<code>off</code> · <code>report</code>	<code>report</code>	At session start, reports when a newer plugin version is checked in than the one installed
<code>orphan_check</code>	<code>off</code> · <code>report</code> · <code>reap</code>	<code>report</code>	At session start, looks for abandoned background Claude workers; <code>reap</code> stops them. A worker is judged by who owns it (0.15.0): a shell with a live <code>claude</code> process above it belongs to that session and is listed as information — session id, claude pid, uptime, and WAITING on: <what it polls> for a poll loop — and is never reaped, even by <code>reap</code> . ORPHAN means no <code>claude</code> remains above it. This session's own undeclared or idle workers are still raised
<code>push_check</code>	<code>off</code> · <code>report</code>	<code>report</code>	At session start, reports vault commits not yet pushed
<code>protected_paths</code>	<code>off</code> · <code>ask</code>	<code>ask</code>	A Write or Edit to the vault's <code>core-rules/</code> or <code>global-memory/</code> , to <code>~/.claude/golden-thread/</code> , or to <code>~/.claude/settings.json</code> always shows the permission prompt; editing an existing file in <code>Sources/</code> is refused (supersede it with a new file). Shell commands that write those files are not seen
<code>test_gate</code>	<code>off</code> · <code>warn</code> · <code>auto</code> · <code>block</code>	<code>auto</code>	Refuse a <code>git commit</code> of code whose tests have not been seen to pass; <code>auto</code> blocks only where the repo has a test command
<code>parallel_work</code>	<code>off</code> · <code>on</code>	<code>on</code>	Whether divisible work runs in parallel at all; <code>off</code> also stops the Core rule being injected
<code>parallel_max</code>	<code>auto</code> · a positive integer	<code>auto</code>	Ceiling on concurrent workers. <code>auto</code> = as many as the machine allows

Settings that come from modules. Since 0.15.0 a module declares its own settings in its `module.json` — `key`, `default`, `values`, `summary`, and an optional long `detail` that `gt_settings.py explain <name>` prints. They are listed under `module <name>` in `show` and exist only while the module is installed. A value you set for a module that is later switched off is kept and shown as *OFF — settings kept, not in effect*; it takes effect again after `bash install.sh --with <module>`.

SETTING	MODULE	VALUES	DEFAULT	WHAT IT DOES
<code>report_card</code>	<code>report-card</code>	<code>off</code> · <code>minimal</code> · <code>full</code>	<code>minimal</code>	At <code>/compact</code> and session end, summarises session hygiene; the card is shown at the start of your next session, because output at those two events is never displayed
<code>closeout_check</code>	<code>report-card</code>	<code>off</code> · <code>ask</code>	<code>ask</code>	At <code>/compact</code> and session end, names a project that looks finished so you are asked whether to close it
<code>watch</code>	<code>watch</code>	<code>off</code> · <code>report</code>	<code>off</code>	<code>/gt-watch:gt-watch</code> : the cron fetch and the session-start report of repo changes

`report_card` (detail):

```
off      nothing
minimal hygiene only -- what went wrong in THIS session  (default)
full     hygiene, plus vault features available and unused
```

Fires on PreCompact so it is produced while there is still context to write it in, rather than competing for the last of it at session end.

closeout_check (detail):

```
off      nothing
ask       name each project whose signals say it may be finished, with the
          reasons, so the assistant puts the question to you  (default)
```

Runs the vault's own `Projects/golden-thread/tools/gt_closeout.py`. Signals: most open tasks past due, most tasks checked off with nothing urgent left, no new task and no write-back for three weeks, or no open task at all. Every time the question is put to you it is recorded with the signal values, and your answer is recorded beside it, so `gt_closeout.py history` can show what 'ready to close' has actually looked like for you and the thresholds can be tuned to that rather than to a guess.

On 2026-09-05 the CYC26 talk had been delivered for two days while 25 of its rehearsal tasks sat open and overdue at the top of the rollup, above live trading work. Nothing had asked.

watch (detail):

```
off      no fetching and no report  (default)
report    the cron fetch runs, and session start lists unacknowledged upstream
          changes: P0s first with repo and reason, review items one line each,
          routine changes as a count
```

A watch is a note in `Projects/golden-thread/watches/`. The fetch runs from cron (`gt_watch.py install-cron`), never from a session; the session-start hook only reads the local queue in `~/claude/golden-thread/watch/`. P0 comes from fixed rules -- a security advisory, a CVE or GHSA id, 'security fix' in a release -- never from a reading of commit prose, because a P0 that cries wolf stops being read. `GT_WATCH=off|report` in the environment overrides this setting.

Heads-up: `parallel_work` is `on` and `parallel_max` is `auto` out of the box, so after upgrading to 0.12.4 a long run will use as much of your machine as it can — many processes at once, CPU well above 100%, and the fans to match. That is intended. Cap it with `set parallel_max <N>` or switch it off with `set parallel_work off`; nothing else about the plugin changes.

Test receipts. `core_test_before_commit` needs evidence that the tests passed on *this* tree, so a run leaves a receipt and the commit guard reads it:

```
gt_test_receipt.py record --repo . --what "pytest -q" --ok    # any project
gt_test_receipt.py latest --repo .                          # what was last recorded
gt_test_receipt.py check  --repo . --files a.py b.py         # does a receipt cover these?
```

A receipt covers a file only if it is newer than that file, so editing something after the run invalidates it automatically. `tests/run.sh` and `dev/release-check.sh` record their own. A repo with no tests is exempted once with `touch .gt-no-test-gate`; a single commit with `GT_TEST_GATE=off`.

The machine profile. `parallel_max: auto` resolves through `parallel_profile`, which `install.sh` measures on this machine and re-measures at every upgrade — cores for CPU-bound work, 2× cores (bounded by memory) for I/O-bound. Inspect or refresh it:

```
gt_settings.py detect-machine      # what this machine would measure
gt_settings.py detect-machine --write  # measure and store it
```

Setting a number above what the machine can use is refused at the point you set it, and a number above core count is accepted with a note that it only helps I/O-bound work.

The budget as a number. Any script, in any project, can ask what the settings allow instead of inventing a worker count:

```
JOBS=$(python3 ../scripts/gt_settings.py jobs 42 --io-bound)  # 42 units of I/O-bound work
JOBS=$(python3 ../scripts/gt_settings.py jobs 8)              # 8 units, CPU-bound
```

It returns 1 when `parallel_work=off`, never more than `parallel_max`, and never more workers than there are units of work. `gt_settings.parallel_jobs()` is the same answer from Python.

```
python3 $SCRIPTS/gt_settings.py show      # current state of all settings
python3 $SCRIPTS/gt_settings.py explain <name> # reasoning behind the default
python3 $SCRIPTS/gt_settings.py set <name> <value>
```

`component_updates` at `auto` will not overwrite an installed file that is newer than the source, and will not delete a file the source lacks — real drift ran that direction, so a naive "source is truth" updater would silently break enforcement on every session start.

`/gt:gt-runbook-lint`

Finds procedures duplicated across runbooks and routes them to the right shared layer — `PROTOCOL.md`, a Knowledge page, or a repo `CLAUDE.md`.

`/gt-watch:gt-watch`

Module `watch` (plugin `gt-watch`), installed by default; it was `/gt:gt-watch` before 0.15.0. `bash install.sh --without watch` removes it, including the crontab line it installed.

Watch any git repo you depend on, and hear about it when it changes in a way you said you care about — raised as a **P0** when it ships a security fix.

```

/gt-watch:gt-watch add <git-url>    - create a watch (any git URL: GitHub, GitLab, self-hosted, file://)
/gt-watch:gt-watch list              - every watch and its last change
/gt-watch:gt-watch check             - fetch now instead of waiting for cron
/gt-watch:gt-watch show <slug>      - Claude reads the new commits and explains them
/gt-watch:gt-watch ack [<slug>]     - mark changes seen
/gt-watch:gt-watch remove <slug>    - stop watching

```

A watch is a note — `Projects/golden-thread/watches/<slug>.md`, editable in Obsidian's Properties panel. Its frontmatter says what to watch: `url`, `track` (commits, tags, releases), `branch`, `watch_paths`, `current_version`, your own `p0_when` patterns, and `starred`.

How it runs. A cron job (`gt_watch.py install-cron --every 1h`) fetches every watch without Claude, classifies each change, and queues it. At session start a hook reads the queue — P0s first, spelled out; review items one line each; routine changes as a count. It never fetches. Machine state lives in `~/.claude/golden-thread/watch/`, outside the vault.

When it is a P0 — decided by rules, never by a model reading commit prose: a new security advisory for the repo; a new tag or release naming a CVE or GHSA id, or saying "security fix/release/update" or "vulnerability"; a commit message carrying a CVE or GHSA id; a change to `SECURITY.md` or your `watch_paths` that says the same; or anything matching your own `p0_when`. A new release, a major-version jump, a change under `watch_paths`, or "BREAKING CHANGE" is a **review** item. Ordinary words like "auth" or "security" in a commit message are not enough — a P0 that cries wolf stops being read.

The module is on by default, but it fetches and reports nothing until you switch it on: setting `watch ( off · report )`. `add` offers to switch it on and install the cron entry.

`/gt-demo:gt-demo`

Module `demo` (plugin `gt-demo`), installed by default. `bash install.sh --without demo` removes it; `--with demo` brings it back.

A guided tour of Golden Thread on PizzaBot 3000, a fictional pizza-ordering project: nine core acts, plus one act for each installed module that ships one — in 0.15.0 `wiki`, `watch` and `flow`, so twelve with the default modules. It runs in its own **throwaway vault**, so nothing it does can reach yours.

```

/gt-demo:gt-demo start    - build the demo vault and print the command that opens it
/gt-demo:gt-demo tour     - (in the demo session) run the tour; you only click Next
/gt-demo:gt-demo end      - every commit and file the tour produced
/gt-demo:gt-demo clean    - delete the demo vault and build a fresh one
/gt-demo:gt-demo remove   - delete the demo vault; `bash install.sh --without demo` removes the module
/gt-demo:gt-demo status   - is there a demo vault, and how old is it

```

Running it: `start` builds the vault at `~/.claude/golden-thread/demo-vault` and prints `cd <demo vault> && GT_VAULT=<demo vault> GT_WATCH=report GT_WATCH_STATE=<demo vault>/.demo/watch claude` — the extra variables let the watch act run without touching this machine's watch settings or state. Run that in a new terminal and type `/gt-demo:gt-demo tour`. For each act Claude says one line to the audience, does the work with the real skill, says what just happened, and offers buttons — **Next**, **Repeat this act**, **Skip ahead**, **End tour**.

The core acts: Core rules enforced (a prepared reply carrying a key is blocked) · open a project · "what's next?" from the task rollout · capture a finding mid-session · route a stray idea · validate a wrong claim · lint finds a planted broken link · promote a finding to a

Knowledge page (which fixes the link) · file the inbox, close the session, and show the receipt. They are plain text in `templates/demo-pizzabot/tour.md` .

Module acts come from each installed module's `module.json` → `demo` file and are placed before the closing act: **wiki** — ingest a source and query the wiki; **watch** — watch an upstream library and see its security release open as a PO; **flow** — render the tour's events as a redacted flow view. A module that is not installed has no act. A module act names its scripts as `<module:NAME>` , which the demo resolves with `gt_demo.sh module-scripts NAME` (exit 3: not installed, the act is skipped).

Why its own vault: a fuller tour writes to shared files (`INBOX.md` , `TASKS.md` , `log.md` , `Knowledge/`) that other sessions also write. Undoing that in a real vault means rewinding history everyone shares. In a throwaway vault, `clean` just rebuilds it. The demo never writes your `vault-config.json` : the demo session is pinned with `GT_VAULT` , which every skill and hook honors. `clean` and `remove` delete a directory only if it carries the demo's marker file.

`/gt-flow:gt-flow`

Module `flow` (plugin `gt-flow`), new in 0.15.0, installed by default.

Draws how knowledge moved through the vault over time, as **one self-contained HTML file** that opens offline — no network, no CDN. One lane per project, time left to right, each lane split into the ladder's levels (memory → research/decisions/design → Knowledge → global-memory → Core); an arrow joins the level an item left to the level it reached, so a promotion is an arrow climbing. It reads

`Projects/golden-thread/events.jsonl` and never writes the vault.

```
FLOW=~/.claude/plugins/cache/golden-thread-plugin/gt-flow/0.15.0/scripts
python3 $FLOW/gt_flow.py render --vault <vault> [--out FILE|DIR] [--redact] \
  [--project <slug> ...] [--since YYYY-MM-DD] [--tasks]
```

- `--redact` before the page leaves your own screen — an Artifact, a chat, a ticket, a screenshot. Every project, path, task id, domain and session becomes a short salted hash and notes are dropped; levels, kinds and counts stay. The salt is random per render, so two redacted files cannot be joined. If the redaction self-check fails, nothing is written.
- Task events (`task.open` , `task.done`) are hidden at first — on a real vault they are most of the stream. They are one click away in the Kinds filter; `--tasks` shows them from the start.
- `--project` is repeatable and includes sub-projects; `--since` takes a date or an ISO-8601 instant.
- Without `--out` the file lands in the current directory (a temp directory when that is inside the vault). An `--out` inside the vault is refused.

Exit codes: `0` written · `1` bad arguments, an invalid event file or an unknown schema version · `2` no events yet (run `gt_events.py backfill --dry-run` to see what history would be recovered) · `3` the filters matched nothing.

Where does this fact go?

THE FACT	GOES TO
"This preset is wrong right now"	memory/
"This API rejects plural keys"	research.md
"We use POST for all mutations, and why"	decisions.md
"The system is structured like this now"	design.md
"This platform behaviour bit two projects"	Knowledge/
"Always diff before overwriting prod"	global-memory/
"How to restart the prod backtest"	runbook.md
"Never scp without backing up"	PROTOCOL.md
"This code lives on host X at path Y"	source.md
"Which box serves which role"	INFRASTRUCTURE.md

Two failure modes, opposite directions: **too narrow** buries a platform truth in one project, so the next rediscovers it the hard way; **too wide** promotes a quirk into `global-memory/`, where it loads into unrelated sessions as a rule that isn't true there. When unsure, file narrow — promotion is cheap, demotion isn't.

`/gt:gt-route` applies this table for you, against the material actually in hand, without your having to hold the table in your head mid-session. Reach for it when the answer is not obvious — including when the honest answer is that the fact belongs to a project other than the one loaded.

Script reference

```
SCRIPTS=~/.claude/plugins/cache/golden-thread-plugin/gt/0.15.0/scripts

python3 $SCRIPTS/vault_init.py fresh --vault ~/my-vault --domain "My Team"

python3 $SCRIPTS/vault_init.py create-project --vault ~/my-vault \
  --name my-project --title "My Project" --tags "backend,api" \
  --domain platform --topology bastion-direct \
  --repo-url git@github.com:me/my-project.git --runbook

python3 $SCRIPTS/vault_init.py create-project --vault ~/my-vault \
  --name sub-feature --parent my-project --domain platform

python3 $SCRIPTS/vault_init.py connect --vault ~/existing-vault

python3 $SCRIPTS/vault_init.py install-core-rules --vault ~/my-vault

python3 $SCRIPTS/gt_ingest.py ~/Projects/my-project --json

python3 $SCRIPTS/gt_lint.py ~/my-vault --queue ~/my-vault/review-queue.md

python3 $SCRIPTS/gt_settings.py show
```

`gt_tasks.py` runs from the vault, at `Projects/golden-thread/tools/`, beside `gt_closeout.py`, `gt_session.py`, `gt_edits.py` and `safe_write.py`. All five ship as templates and are seeded there by `vault_init.py` fresh or `connect` (what `/gt:gt-init` runs); `install.sh` refreshes copies that predate the installed template. It regenerates `TASKS.md` from every project's `## Tasks` section.

```
python3 ~/my-vault/Projects/golden-thread/tools/gt_tasks.py
```

It infers the vault from its own location, so it needs no arguments; `--vault` is there to override that. **Re-run it before reading `TASKS.md`** — project priority is computed against the clock (stale-P1 ageing, deadline windows, `pp_escalate`), so the ranking changes with time even when no file has changed. `TASKS.md` is generated and must never be hand-edited.

`--json` prints the ranked rollup to stdout without writing `TASKS.md`. Deadline windows may span days (0.13.0): `Fri 16:00 → Sun 16:44 America/Chicago → 0`, including spans that wrap the week; the zone always comes from the rule, never the machine.

`gt_lint.py --json` emits findings as JSON, and `gt_lint.py --runbooks` reports lines duplicated across projects' runbooks — the detection step of `/gt:gt-runbook-lint`.

Troubleshooting

"vault-config.json not found" — run `/gt:gt-init`.

A rule in `CLAUDE.md` seems ignored — it is read at session start. Added mid-session, it applies from the *next* session. Same for hooks in `settings.json`: open `/hooks` once or restart.

Memory files aren't being read — that is intended. `gt-open` reads `MEMORY.md` and loads files on demand. Ask for one by name.

A memory file is never offered — run `/gt:gt-lint`; it is probably `memory-unlisted`, so nothing knows it exists.

`/gt:gt-query` can't find a page you added — likely `index-gap`.

Lint suppression isn't working — paths are matched case-insensitively against the finding path; make sure you used the path as the finding reports it.

"conflict: vault-config.json points to a different vault" — run `vault_init.py connect --vault <new-path>`.

core-unenforced finding in lint — Core rules are stored but not being asserted. Run `vault_init.py install-core-rules --vault <vault>` or wire the hooks by hand. Do not suppress this finding.